

7. BÚSQUEDAS INSPIRADAS EN LA NATURALEZA

(PARTE 1)

IA 3.2 - Programación III

1º C - 2026

Dr. Mauro Lucci



La naturaleza como fuente de inspiración

— — —

- Simular cómo la naturaleza resuelve ciertos problemas.
- Algoritmos inspirados en fenómenos biológicos, leyes físicas, procesos químicos, etc.
- Familia muy amplia de algoritmos.
- Han demostrado ser muy efectivos en problemas de optimización.
- Al día de hoy, sigue siendo un área muy productiva en el diseño de algoritmos.

Gran
variedad de
algoritmos
de búsqueda
inspirados
en la
naturaleza.

Swarm intelligence based algorithms		Bio-inspired (not SI-based) algorithms	
Algorithm	Author	Algorithm	Author
Accelerated PSO	Yang et al.	Atmosphere clouds model	Yan and Hao
Ant colony optimization	Dorigo	Biogeography-based optimization	Simon
Artificial bee colony	Karaboga and Basturk	Brain Storm Optimization	Shi
Bacterial foraging	Passino	Differential evolution	Storn and Price
Bacterial-GA Foraging	Chen et al.	Dolphin echolocation	Kaveh and Farhoudi
Bat algorithm	Yang	Japanese tree frogs calling	Hernández and Blum
Bee colony optimization	Teodorović and Dell'Orco	Eco-inspired evolutionary algorithm	Parpinelli and Lopes
Bee system	Lucic and Teodorovic	Egyptian Vulture	Sur et al.
BeeHive	Wedde et al.	Fish-school Search	Lima et al.
Wolf search	Tang et al.	Flower pollination algorithm	Yang
Bees algorithms	Pham et al.	Gene expression	Ferreira
Bees swarm optimization	Drias et al.	Great salmon run	Mozaffari
Bumblebees	Comellas and Martinez	Group search optimizer	He et al.
Cat swarm	Chu et al.	Human-Inspired Algorithm	Zhang et al.
Consultant-guided search	Iordache	Invasive weed optimization	Mehrabian and Lucas
Cuckoo search	Yang and Deb	Marriage in honey bees	Abbass
Eagle strategy	Yang and Deb	OptBees	Maia et al.
Fast bacterial swarming algorithm	Chu et al.	Paddy Field Algorithm	Premaratne et al.
Firefly algorithm	Yang	Roach infestation algorithm	Havens
Fish swarm/school	Li et al.	Queen-bee evolution	Jung
Good lattice swarm optimization	Su et al.	Shuffled frog leaping algorithm	Eusuff and Lansey
Glowworm swarm optimization	Krishnanand and Ghose	Termite colony optimization	Hedayatzadeh et al.
Hierarchical swarm model	Chen et al.	Physics and Chemistry based algorithms	
Krill Herd	Gandomi and Alavi	Big bang-big Crunch	Zandi et al.
Monkey search	Mucherino and Seref	Black hole	Hatamlou
Particle swarm algorithm	Kennedy and Eberhart	Central force optimization	Formato
Virtual ant algorithm	Yang	Charged system search	Kaveh and Talatahari
Virtual bees	Yang	Electro-magnetism optimization	Cuevas et al.
Weightless Swarm Algorithm	Ting et al.	Galaxy-based search algorithm	Shah-Hosseini
Other algorithms		Gravitational search	Rashedi et al.
Anarchic society optimization	Shayeghi and Dadashpour	Harmony search	Geem et al.
Artificial cooperative search	Civicioglu	Intelligent water drop	Shah-Hosseini
Backtracking optimization search	Civicioglu	River formation dynamics	Rabanal et al.
Differential search algorithm	Civicioglu	Self-propelled particles	Vicsek
Grammatical evolution	Ryan et al.	Simulated annealing	Kirkpatrick et al.
Imperialist competitive algorithm	Atashpaz-Gargari and Lucas	Stochastic diffusion search	Bishop
League championship algorithm	Kashan	Spiral optimization	Tamura and Yasuda
Social emotional optimization	Xu et al.	Water cycle algorithm	Eskandar et al.

Recocido simulado

Simulated Annealing (SA)

El **recocido simulado** es un algoritmo de **búsqueda local** que utiliza un mecanismo **probabilístico**, regulado por un parámetro de **temperatura**, para **escapar** de máximos locales.

— — —

Recocido

— — —

El **recocido** es un tratamiento térmico que permite alterar la estructura molecular de un material para mejorar sus propiedades mecánicas (ductilidad, dureza, tensiones internas).



1. **Calentar** un material por encima de su temperatura de recristalización.
2. **Mantener** la temperatura por un período de tiempo
3. **Enfriar** progresivamente hasta alcanzar la temperatura ambiente.

Recocido simulado

— — —

- Tiene una **temperatura** que se enfría con el paso de las iteraciones.
- Puede moverse a un vecino que empeore la función objetivo del estado actual con una cierta **probabilidad**.
- Esta probabilidad decrece a medida que la temperatura baja.
 - Los “malos” movimientos son más probables al comienzo y se vuelven menos probables en las últimas iteraciones.
- Maximizamos una **función objetivo** f .

Recocido simulado

— — —

Idea.

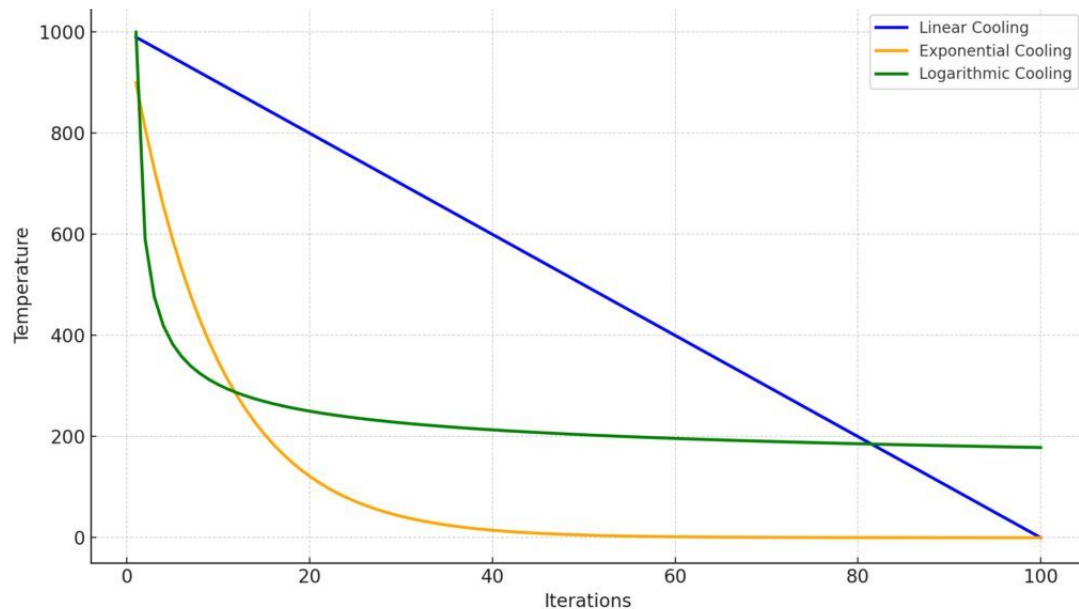
1. Comienza en un estado inicial.
2. En cada iteración:
 - Se **actualiza** la temperatura.
 - Se elige al **azar** un estado sucesor s' del estado actual s .
 - Se acepta s' como nuevo estado actual con una cierta **probabilidad**.
3. El algoritmo se detiene cuando la temperatura alcanza un valor final, típicamente un valor cercano 0.

Esquema de recocido.

Función de probabilidad de aceptación.

Esquema de recocido o enfriamiento

El **esquema de recocido** es una función que determina la temperatura en función de número de iteraciones.



Tipos de enfriamiento

Es el más
utilizado

Lineal.

$$\begin{cases} t_0 \\ t_{i+1} = t_i - \alpha \\ \quad = t_0 - (i+1) \cdot \alpha \end{cases}$$

$\alpha > 0$ constante

**Geométrico o
exponencial.**

$$\begin{cases} t_0 \\ t_{i+1} = \beta \cdot t_i \\ \quad = \beta^{i+1} \cdot t_0 \end{cases}$$

$0 < \beta < 1$ constante
(factor de
enfriamiento)

Logarítmico.

$$t_i = Y / \log(i+1)$$

$Y > 0$ constante

Función de probabilidad de aceptación

— — —

Dado un estado s , un sucesor s' de s y una temperatura t , la función de **probabilidad de aceptación** determina la probabilidad de reemplazar a s' por s como nuevo estado actual cuando la temperatura es t .

Función de probabilidad de aceptación

En la práctica...
¿Qué función se
suele usar? 🤔

Para todo s , s' y t , una función de probabilidad de aceptación P debe verificar:

1. Si $f(s) < f(s')$, entonces $P(s, s', t) = 1$

independientemente de la temperatura.

Si s' es mejor que s , se acepta siempre

2. Si $f(s) \geq f(s')$, entonces $0 < P(s, s', t) \leq 1$

y tiende a 0 cuando t disminuye.

Sino, la probabilidad de aceptar s' depende de la temperatura

Función de probabilidad de aceptación

— — —

Típicamente, se usa como función de probabilidad de aceptación la siguiente función **exponencial**:

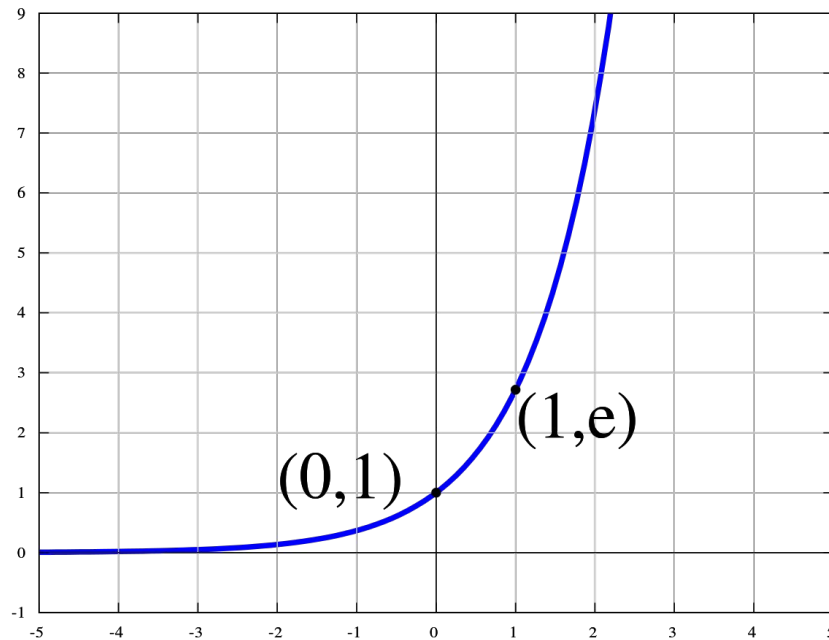
$$P(s, s', t) = e^{\frac{f(s') - f(s)}{t}}$$

¿Por qué se usa esta función?

Función $f(x) = e^x$

En la función $f(x) = e^x$

- Si $x > 0$, $e^x > 1$.
- Si $x \leq 0$, $e^x \in (0,1]$.
- Es continua y creciente.



Gráfica de e^x

Función de probabilidad de aceptación

$$P(s, s', t) = e^{\frac{f(s') - f(s)}{t}}$$

1. Si $f(s) < f(s')$, exponente positivo y $P(s, s', t) > 1$.
2. Si $f(s) \geq f(s')$, exponente no-positivo y $0 < P(s, s', t) \leq 1$.



Ejemplos

— — —

$$P(s, s', t) = e^{\frac{f(s') - f(s)}{t}}$$

Para $f(s')=8$ y $f(s)=10$,

- $P(s, s', 100) = 98\%$.
- $P(s, s', 10) = 82\%$.
- $P(s, s', 2) = 37\%$.

Fijado el estado y el sucesor, la probabilidad de aceptar el sucesor baja con la temperatura



Ejemplos

— — —

$$P(s, s', t) = e^{\frac{f(s') - f(s)}{t}}$$

Para $f(s)=10$ y $t=10$,

- Si $f(s')=8$, $P(s, s', 100) = 82\%$.
- Si $f(s')=6$, $P(s, s', 10) = 67\%$.
- Si $f(s')=4$, $P(s, s', 2) = 55\%$.

*Fijado el estado y la temperatura,
la probabilidad de aceptar un
sucesor es menor mientras peor
sea el sucesor*



Algoritmo de recocido simulado

```
1 function RECOCIDO-SIMULADO(problema, esquema, P) return estado
2   actual ← problema.estado-inicial
3   for i = 0 to ∞
4     t ← esquema(i)
5     if t es cercano a 0 then return actual
6     sucesor ← sucesor de actual elegido al azar
7     actual ← sucesor with probability P(actual, sucesor, t)
```

¿Cómo aceptar con probabilidad? 🤔



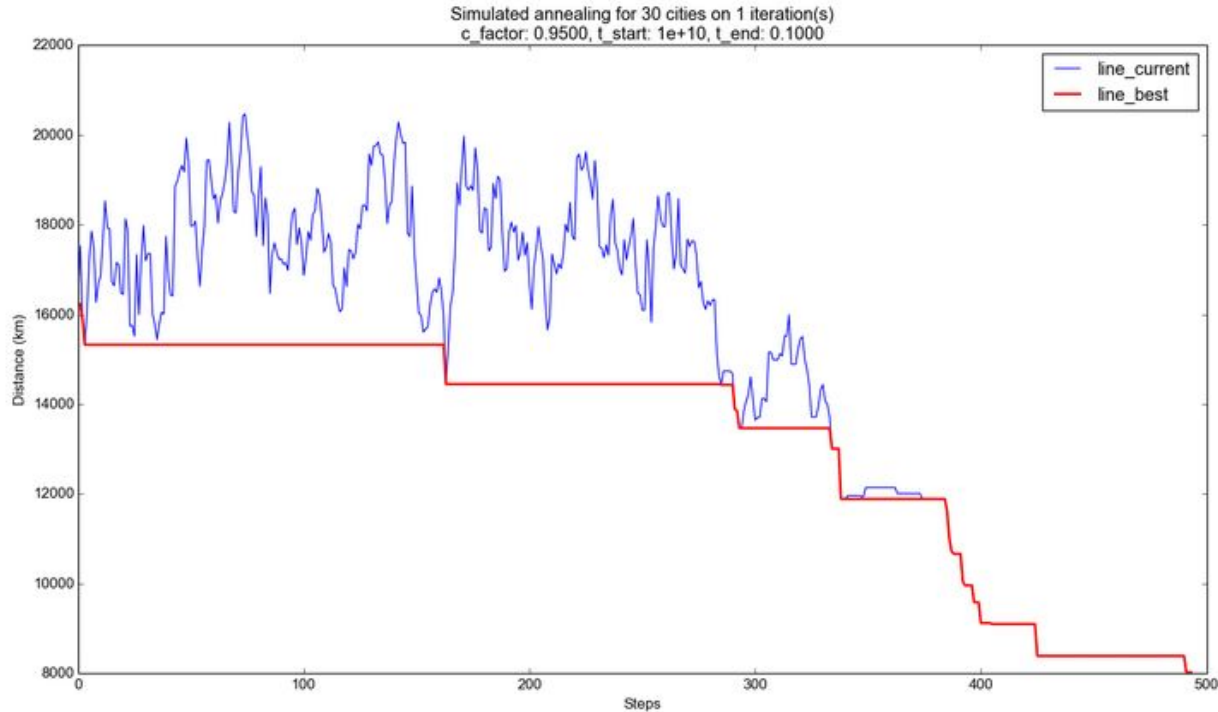
Algoritmo de recocido simulado

```
1 function RECOCIDO-SIMULADO(problema, esquema, P) return estado
2   actual ← problema.estado-inicial
3   for i = 0 to ∞
4     t ← esquema(i)
5     if t es cercano a 0 then return actual
6     sucesor ← sucesor de actual elegido al azar
7     if random(0,1) ≤ P(actual, sucesor, t) then
8       actual ← sucesor
```

Para decidir si aceptamos a un sucesor con probabilidad p , generamos un número aleatorio r en $[0,1]$. Aceptamos el sucesor si y sólo si $r \leq p$.



Ejemplo



En las primeras iteraciones, la gráfica azul da grandes saltos.

Las gráficas tienden a equipararse hacia el final de la ejecución y el descenso se acelera.

Número de iteraciones vs. valor objetivo del estado actual (azul) y del mejor estado (rojo) para un TSP de 30 ciudades resuelto con SA.



Ejercicio

Considerar un algoritmo de **recocido simulado** con un esquema de recocido exponencial $t_{i+1} = \beta^{i+1} \cdot t_0$ y la función de probabilidad de aceptación $P(s, s', t) = e^{\frac{f(s') - f(s)}{t}}$.

Se usa una temperatura inicial $t_0 = 1000$ y un factor de enfriamiento $\beta = 0,99$.

Siendo un estado s con $f(s) = 200$ en la iteración $i = 100$, determinar la probabilidad de aceptar un sucesor s' de s con $f(s') = 75$.



Respuesta

— — —

$$t_0 = 1000$$

$$t_1 = \beta \cdot t_0 = 990$$

$$t_2 = \beta \cdot t_1 = \beta \cdot (\beta \cdot t_0) = \beta^2 \cdot t_0 = 980$$

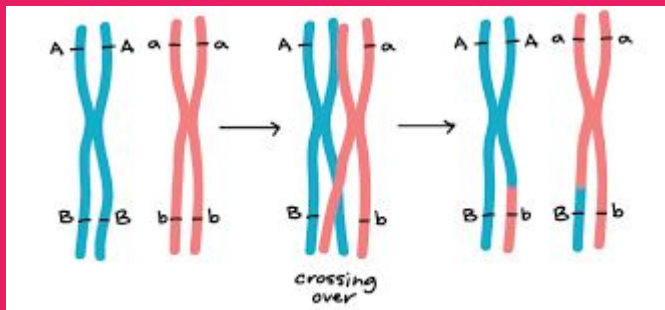
...

$$t_{100} = \beta^{100} \cdot t_0 = 0,99^{100} \cdot 1000 = 366$$

$$P(s, s', t_{100}) = e^{\frac{f(s') - f(s)}{t_{100}}} = e^{\frac{75 - 200}{366}} = 71\%$$

Algoritmo Genético

Genetic Algorithm (GA)



Los **algoritmos genéticos** se inspiran en la teoría evolutiva de la **selección natural**.

- Generación inicial con con una **población** de individuos.
- En cada iteración, se **seleccionan** los mejores individuos de la generación actual.
- Los genomas de los individuos seleccionados se **cruzan** y se **mutan** para formar la siguiente generación.
- Típicamente, el algoritmo termina cuando se alcanza cierto número de generaciones.

— — —

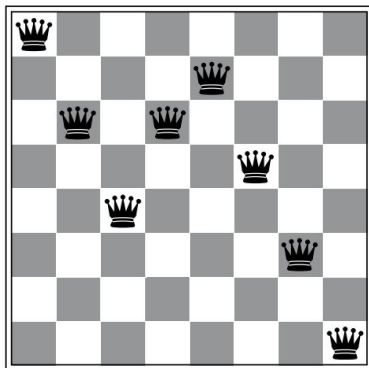
1. Población inicial

Un GA comienza con una **población inicial** formada por un conjunto de k **individuos** (estados) generados aleatoriamente.

El genoma de cada individuo se representa con una string sobre un alfabeto finito.



Ejemplo (8-Reinas).



Genoma

1 3 5 3 2 4 6 8

El i -ésimo dígito es la fila en la que se encuentra la reina de la columna i .

2. Función de fitness

— — —

Cada individuo se califica con una **función de fitness** (función objetivo).

La función de fitness (para un problema de maximización) debe devolver valores más altos para los mejores individuos.



Ejemplo (8-Reinas).

Una función de fitness puede ser el número de pares de reina que no se atacan.

Los estados objetivo tienen un valor de 28 (formas de tomar 2 elementos de un conjunto de 8).

3. Selección

— — —

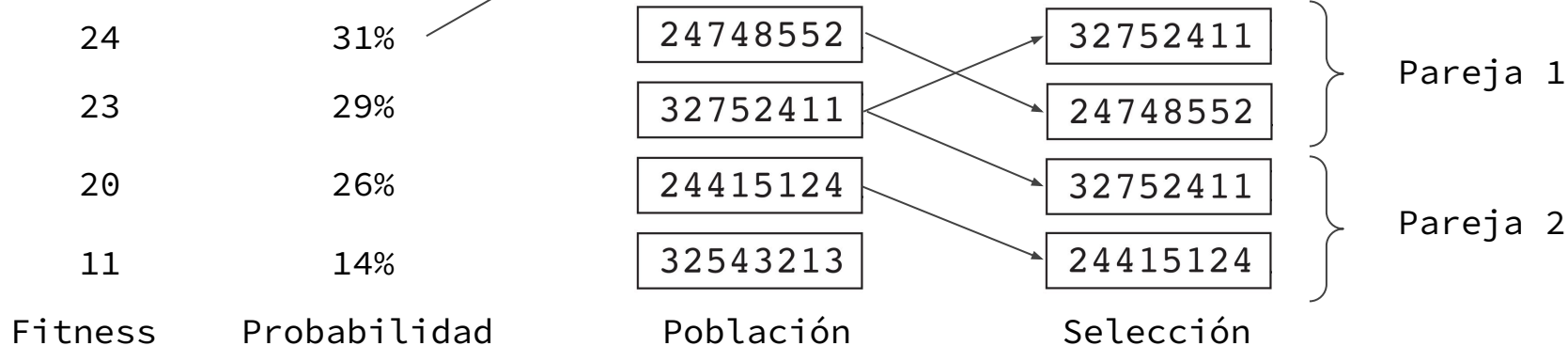
La **selección** consiste en elegir aleatoriamente pares de individuos de la generación actual para reproducirlos, pero con una distribución no-uniforme.

Usualmente, la probabilidad de que un individuo sea elegido es directamente **proporcional** a su valor de fitness.



Ejemplo (8-Reinas).

$$31\% = 24 / (24 + 23 + 20 + 11)$$



3. Selección

¿Cómo elegir aleatoriamente a un individuo con probabilidades diferenciadas?

31%	24748552
29%	32752411
26%	24415124
14%	32543213
Probabilidad	Población

Generemos un número aleatorio r en $[0,1]$. Luego, si...

- $0 \leq r \leq 0.31$, individuo 1.
- $0.31 < r \leq 0.60$, individuo 2.
- $0.60 < r \leq 0.86$, individuo 3.
- $0.86 < r \leq 1$, individuo 4.

4. Cruzamiento

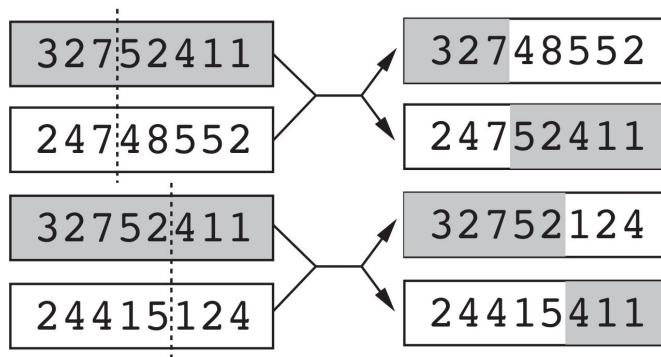
— — —

El **cruzamiento** consiste en generar nuevos individuos a partir de la información genética de sus padres.

Cruzamiento clásico. Se elige una posición i aleatoria de la string y se combinan los primeros i caracteres del primer individuo con los caracteres restantes del segundo individuo.

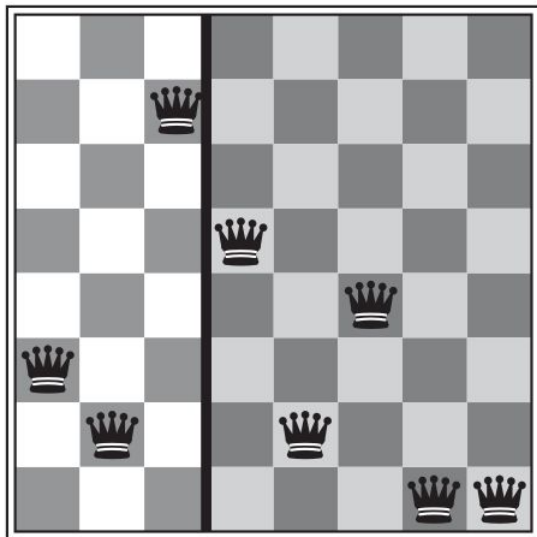


Ejemplo (8-Reinas).

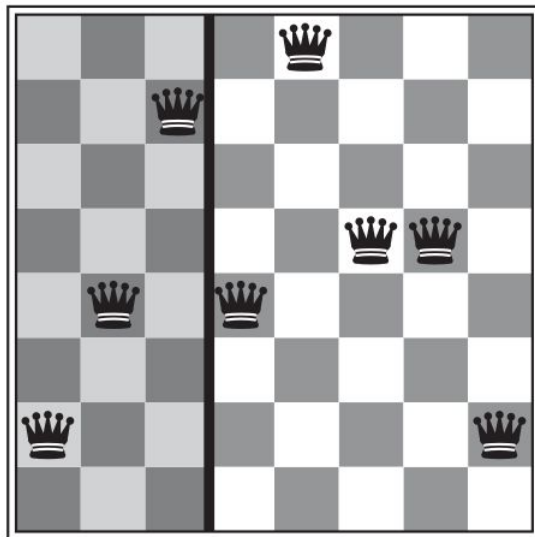


4. Cruzamiento

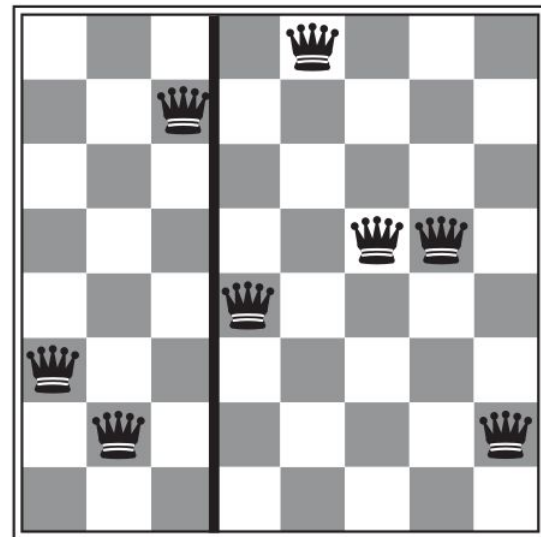
— — —



+



=



5. Mutación

La **mutación** consiste en realizar con una probabilidad baja un cambio aleatorio en la información genética del nuevo individuo.

Mutación clásica. Se elige una posición i aleatoria de la string y se cambia su valor por un valor aleatorio.



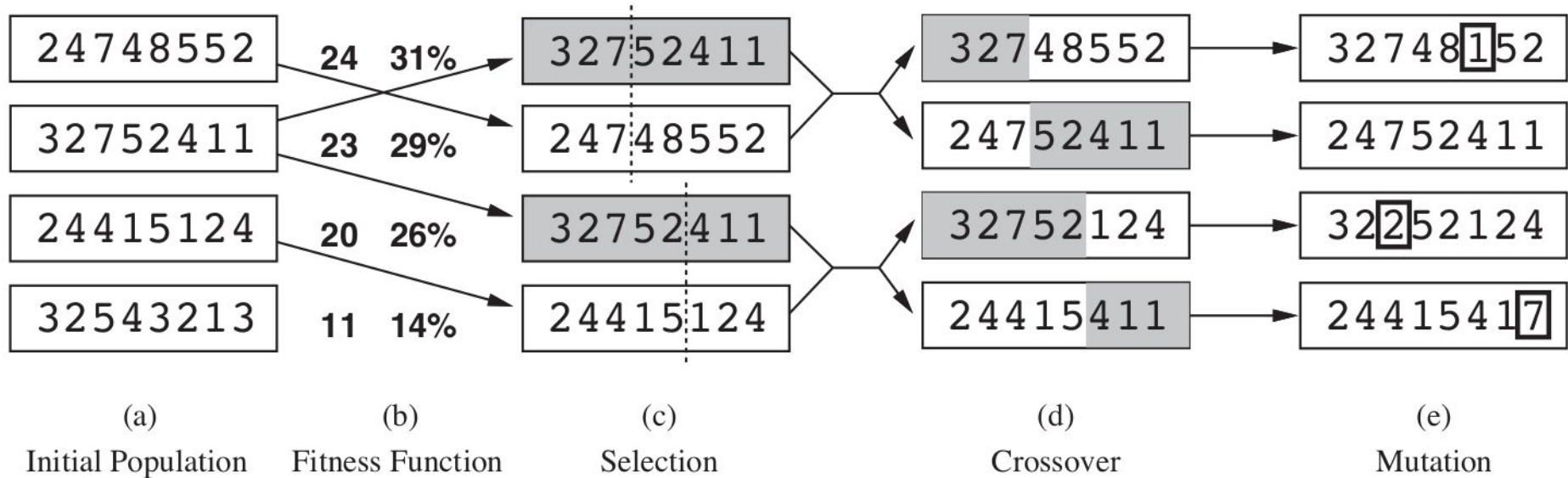
Ejemplo (8-Reinas).



Esta mutación representa mover a la reina de la columna 6 de la fila 5 a la 1.



Ejemplo (8-Reinas)

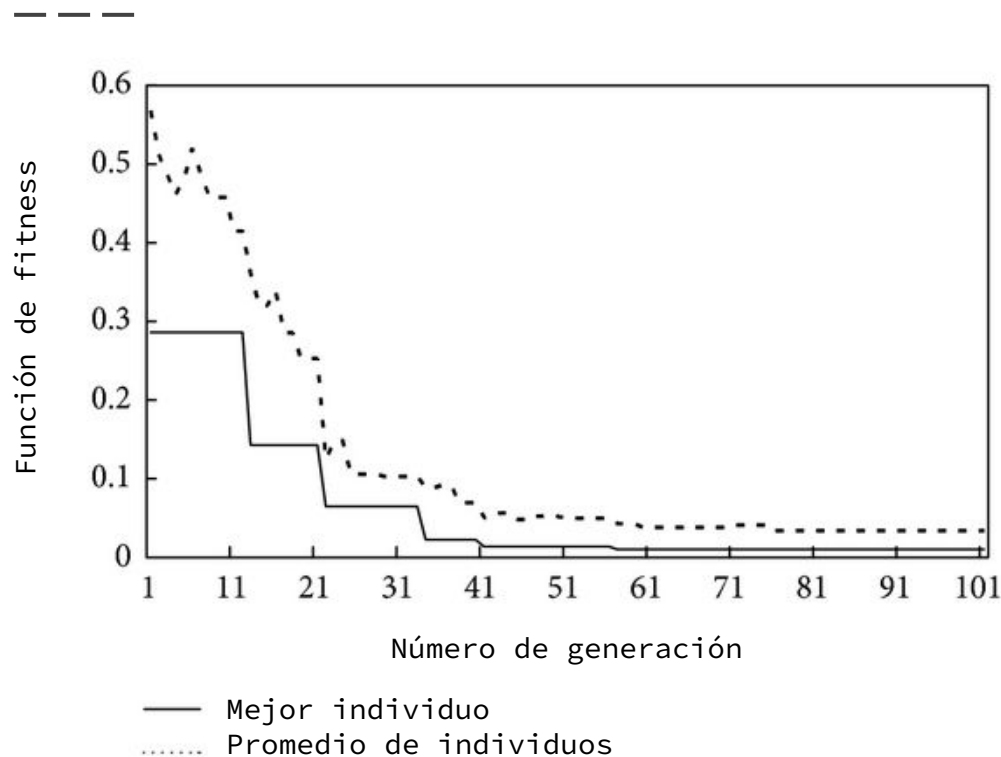




Algoritmo genético

```
1 function ALGORITMO-GENÉTICO(población, fitness) return individuo
2   while no se cumple el criterio de parada do
3     población' ← {}
4     for i = 1 to len(población)
5       x ← SELECCIÓN(población, fitness)
6       y ← SELECCIÓN(población, fitness)
7       hijo ← CRUZAR(x, y)
8       if probabilidad baja then hijo ← MUTAR(hijo)
9       población' ← población' U {hijo}
10    población ← población'
11  return argmax(fitness, población)
```

Evolución de las generaciones



Convergencia de la población.

Inicialmente, la población es bastante **diversa**, algunos individuos son buenos y otros malos.

A medida que avanzan las generaciones, se pierde diversidad genética. La selección favorece reproducir individuos de mayor calidad. Las características de las mejores soluciones se propagan en la población.



Ejercicio

— — —

Dar las componentes necesarias para un **algoritmo genético** que resuelva el TSP.



Respuesta

- **Representación de camino.** Con strings sobre el alfabeto $\{1, \dots, n\}$ según el orden en que se visitan las ciudades.

Por ejemplo, el tour $2 \rightarrow 3 \rightarrow 1 \rightarrow 5 \rightarrow 4$ sería la string “2 3 1 5 4”

- **Función de fitness.** Costo (distancia total) del tour.
- **Cruzamiento.** El cruzamiento clásico puede romper el tour.

1 3 | 4 2 5 1 3 5 1 2

4 3 | 5 1 2 4 3 4 2 5

Podemos arreglarlo cambiando ciudades repetidas por ciudades no visitadas.

1 3 5 1 2 $\rightarrow$ 1 3 5 4 2

4 3 4 2 5 $\rightarrow$ 4 3 1 2 5

- **Mutación.** La mutación clásica rompe el tour. En su lugar, podemos aplicar un movimiento de 2-opt entre dos arcos aleatorios del tour
[\[https://en.wikipedia.org/wiki/2-opt#/media/File:2-opt_wiki.svg\]](https://en.wikipedia.org/wiki/2-opt#/media/File:2-opt_wiki.svg).

¿Existe un genoma compatible con el cruzamiento y la mutación clásicas?



Respuesta

Representación ordinal.

Permite usar el cruzamiento y la mutación clásicos.

Dado el orden

Orden = 1 2 3 4 5

y el tour

3 → 5 → 2 → 4 → 1

¿Cómo lo representamos?

Tour	Orden	Repr.
3	1 2 3 4 5	3

El 1er elemento de Tour es 3 y está 3ro en Orden, entonces lo representamos con un 3.

Sacamos a 3 de Orden y seguimos...



Respuesta

Representación ordinal.

Permite usar el cruzamiento y la mutación clásicos.

Dado el orden

Orden = 1 2 3 4 5

y el tour

3 → 5 → 2 → 4 → 1

¿Cómo lo representamos?

Tour	Orden	Repr.
3	1 2 3 4 5	3
5	1 2 4 5	4

El 2do elemento de Tour es 5 y está 4to en el nuevo Orden, entonces lo representamos con un 4.

Sacamos a 5 de Orden y seguimos...



Respuesta

Representación ordinal.

Permite usar el cruzamiento y la mutación clásicos.

Dado el orden

Orden = 1 2 3 4 5

y el tour

3 → 5 → 2 → 4 → 1

¿Cómo lo representamos?

Tour	Orden	Repr.
3	1 2 3 4 5	3
5	1 2 4 5	4
2	1 2 4	2



Respuesta

Representación ordinal.

Permite usar el cruzamiento y la mutación clásicos.

Dado el orden

Orden = 1 2 3 4 5

y el tour

3 → 5 → 2 → 4 → 1

¿Cómo lo representamos?

Tour	Orden	Repr.
3	1 2 3 4 5	3
5	1 2 4 5	4
2	1 2 4	2
4	1 4	2



Respuesta

Representación ordinal.

Permite usar el cruzamiento y la mutación clásicos.

Dado el orden

Orden = 1 2 3 4 5

y el tour

3 → 5 → 2 → 4 → 1

¿Cómo lo representamos?

Tour	Orden	Repr.
3	1 2 3 4 5	3
5	1 2 4 5	4
2	1 2 4	2
4	1 4	2
1	1	1

Llegamos a la representación 3 4 2 2 1.



Respuesta

— — —

Otras representaciones y operadores de cruzamiento y mutación para el TSP que existen en la literatura.

Representation	Operators	Authors
<i>Binary</i>	Classical + repair operator	Lidd (1991)
<i>Path</i>	Partially-mapped crossover Order-crossover Order based crossover Position based crossover Heuristic crossover Edge recombination crossover Sorted match crossover Maximal preservative crossover Voting recombination crossover Alternating-positions crossover Displacement mutation Exchange mutation Insertion mutation Simple inversion mutation Inversion mutation Scramble mutation	Goldberg and Lingle (1985) Davis (1985) Syswerda (1991) Syswerda (1991) Grefenstette (1987b) Whitley et al. (1989) Brady (1985) Mühlenbein et al. (1988) Mühlenbein (1989) Larrañaga et al. (1996a) Michalewicz (1992) Banzhaf (1990) Fogel (1988) Holland (1975) Fogel (1990) Syswerda (1991)
<i>Adjacency</i>	Alternating edge crossover Subtour chunks crossover Heuristic crossover 1 Heuristic crossover 2 Heuristic crossover 3	Grefenstette et al. (1985) Grefenstette et al. (1985) Grefenstette et al. (1985) Jog et al. (1989) Suh and Van Gucht (1987)
<i>Ordinal</i>	Classical operators	Grefenstette et al. (1985)
<i>Matrix</i>	Intersection crossover operator Union crossover operator Repair operators Repair operators Heuristic inversion mutation	Fox and McMahon (1987) Fox and McMahon (1987) Soni (1981) Homaifar and Guan (1991) Homaifar and Guan (1991)